

dMart UCI

Sistema de Gestion Integral de la Unidad de Cuidados Intensivos

INFORME TECNICO

Estado Actual, Capabilidades y Roadmap del Sistema

Proyecto: dMart UCI — Plataforma de Gestion UCI

Version: v0.5.0 (Fase 0-5 completadas)

Fecha: 14 de September de 2026

Institucion: Universidad Central de Venezuela

Clasificacion: Confidencial — Uso academico interno

*Elaborado para revision por asesores academicos y directivos
conteniendo directivos clinicos y doctor en matematicas*

Indice

1. Resumen Ejecutivo
2. Objetivos del Sistema
3. Stack Tecnológico
4. Arquitectura de Software
5. Estado Actual de Desarrollo
6. Modelos de Datos y Dominio Clínico
7. Sistemas de Puntuación Clínica
8. Motor de Predicción (Machine Learning)
9. Interoperabilidad (HL7 / FHIR)
10. Seguridad y Cumplimiento Normativo
11. Infraestructura y Despliegue
12. Calidad de Software y Testing
13. Qué Falta para Producción Hospitalaria
14. Análisis de Riesgos
15. Cronograma y Proyección
16. Conclusiones y Recomendaciones

1. Resumen Ejecutivo

dMart UCI es una plataforma integral de gestion para Unidades de Cuidados Intensivos (UCI) desarrollada enteramente en Rust, uno de los lenguajes de programacion con mayor garantia de seguridad y rendimiento en la industria del software. El sistema esta disenado para operar en entornos hospitalarios reales, ofreciendo calculo automatizado de escalas de severidad clinica, prediccion de mortalidad mediante inteligencia artificial, interoperabilidad con estandares internacionales (HL7 v2, FHIR R4), y un panel de administracion completo.

A la fecha de este informe (septiembre 2026), el sistema se encuentra en la Version 0.5.0 con las Fases 0 a 5 completamente finalizadas (de un total de 8 fases planificadas). Esto representa la finalizacion del nucleo funcional del sistema: seguridad, datos, operaciones, interfaz de usuario, y validacion clinica con pruebas avanzadas. El sistema ya cuenta con 136+ tests automatizados, integracion con monitores de cama via protocolo HL7, exportacion en PDF/CSV, y un frontend moderno compilado a WebAssembly.

El proyecto sigue la metodologia Spec-Driven Development (SDD), donde cada funcionalidad nace de una especificacion formal con criterios de aceptacion antes de escribir codigo. De 31 especificaciones planificadas, 10 estan completadas y 2 adicionales estan listas para ejecucion. Las fases restantes (6-8) abarcan hardening, CI/CD, Machine Learning avanzado, y despliegue en produccion.

Metricas Clave del Proyecto

Version v0.5.0	Fases 6/8 (75%)	Tests 136+	Specs 10/31
Cobertura HL7 >90%	Rendimiento ~4 ns	Tamano WASM 2.2 MB	Imagen Docker 48 MB

2. Objetivos del Sistema

2.1 Triple Objetivo Estrategico

- **Produccion hospitalaria real:** El sistema debe operar en una UCI con cero incidentes de seguridad y datos clinicos intactos. El criterio de exito es sobrevivir 30 dias en una UCI de 5 camas sin intervencion directa del desarrollador.
- **Producto comercializable:** Despliegue reproducible con Docker, monitoreo con Grafana/Prometheus, soporte y auditoria alineada con estandares HIPAA/NIST/ISO 27001.
- **Portfolio academico:** Arquitectura limpia documentada, tests exhaustivos, metricas de calidad verificables, y un modelo de Machine Learning explicable.

2.2 Funcionalidades Principales

- **Gestion de pacientes UCI:** Registro demografico completo, ingreso/egreso con desenlace clinico (Mejorado/Trasladado/Fallecido), historial y evolucion temporal.
- **Scores clinicos en tiempo real:** APACHE II, Glasgow Coma Scale (GCS), NEWS2, SOFA, SAPS III — todos calculados automaticamente al registrar mediciones.
- **Prediccion de mortalidad:** Modelo de Machine Learning (Decision Tree) entrenado con datos sinteticos basados en reglas clinicas de APACHE II. Precision estimada: 85-90%.
- **Monitoreo de monitores de cama:** Ingesta automatica de datos desde monitores Mindray y Philips via protocolo HL7 v2 (ORU^R01) sobre MLLP/TCP.
- **Interoperabilidad FHIR R4:** Exposicion de pacientes, observaciones, diagnosticos (CIE-10) y reportes con codigo QR en formato FHIR R4.
- **Dashboard ejecutivo:** KPIs de unidad: mortalidad predicha vs real, estancia promedio, ocupacion de camas, distribucion de severidad.
- **Seguridad hospitalaria:** Autenticacion multifactor (TOTP), RBAC por rol (Admin/Medico/Enfermero/Viewer), auditoria de acceso con retencion de 6 anos.
- **Operacion offline:** Progressive Web App (PWA) con soporte offline via Service Worker, base de datos embebida, sin dependencia de internet.

3. Stack Tecnológico

dMart UCI esta construido sobre un ecosistema 100% Rust, aprovechando las garantias de memoria segura, rendimiento nativo, y ausencia de runtime de garbage collector. Esta decision tecnica es critica para software clinico donde los crashes y memory leaks pueden tener consecuencias directas sobre pacientes.

Componentes del Stack

Capa	Tecnologia	Version	Justificacion
Lenguaje	Rust	1.98	Memoria segura, cero costos de abstraccion
Backend	Axum + Tokio	0.8	Async de alta concurrencia, ecosistema maduro
Frontend	Leptos + WASM	0.8	UI reactiva compila a WebAssembly nativo
Base datos	SurrealDB (SurrealKV)	2.x	Embebida, grafos + documentos, Sin SQL compleja
Cache	Valkey / Redis	8+	Sesiones, rate-limit, revocacion JWT
ML	linfa + ndarray	0.7	Decision Trees nativos Rust, sin Python
QA	cargo-fuzz, proptest	-	Property-based testing + fuzzing
Load test	k6	0.54	Escenarios de carga hasta 1000 req/s
E2E	Playwright	-	15 tests automatizados de extremo a extremo
Contenedores	Docker multi-stage	-	Imagen final ~48MB, Debian Bookworm Slim

Decision Arquitectonica: Rust sobre Alternativas

La seleccion de Rust (en lugar de Java, Python o C#) se justifica por tres razones clinico-tecnicas: (1) Ausencia de garbage collector elimina pausas impredecibles criticas en sistemas de monitoreo en tiempo real; (2) El modelo de ownership previene memory leaks y data races en compile-time, reduciendo la superficie de bugs en software que maneja datos de pacientes; (3) El rendimiento nativo (~4 nanosegundos para APACHE II) permite calcular scores en el navegador del clinico sin latencia perceptible, incluso en hardware clinico de gama baja.

4. Arquitectura de Software

4.1 Arquitectura General

El sistema sigue una arquitectura de tres capas con separación estricta:

- **Capa de Presentación (Frontend):** Aplicación Leptos compilada a WebAssembly. Rendering CSR (Client-Side Rendering) con PWA. Comunica con el backend via HTTP REST y recibe actualizaciones en tiempo real via Server-Sent Events (SSE). Estado reactivo con signals de Leptos, persistencia local en LocalStorage, cache TTL de 60s.
- **Capa de Negocio (Backend):** Servidor Axum async con middleware de autenticación JWT, autorización RBAC, rate-limiting por IP, y throttle de login. Expone ~40 endpoints REST, un stream SSE, y un servidor TCP para recepción HL7 MLLP. Cada request pasa por la cadena: rate-limit -> login-throttle -> auth-middleware -> RBAC -> handler.
- **Capa de Persistencia:** SurrealDB embebida (SurrealKV) como base de datos principal con esquema versionado (migraciones SurrealQL). Valkey/Redis como cache distribuido para sesiones, revocación de tokens, y estado del rate-limiter. Sin dependencia de servicios externos para el modo embebido.

4.2 Módulos del Backend (17 módulos)

- **auth.rs:** Autenticación JWT con refresh tokens rotativos, Argon2id, MFA TOTP
- **rbac.rs:** Control de acceso basado en roles: Admin, Medico, Enfermero, Viewer
- **security.rs:** Rate-limiting token-bucket, throttle de login, headers HSTS, sanitización
- **mfa.rs:** TOTP con códigos de respaldo (10 códigos alfanuméricos de 8 caracteres)
- **crypto.rs:** Cifrado ChaCha20-Poly1305 con derivación de clave maestra via Argon2id KDF
- **db.rs:** Capa de acceso a SurrealDB: CRUD completo, búsqueda full-text, transacciones atómicas
- **api/:** 12 módulos de endpoints: auth, patients, measurements, scales, fhir, admin, export...
- **hl7/:** Parser HL7 v2 (ORU^R01), detección de fabricante (Mindray/Philips), clasificación LOINC
- **ingest/:** Pipeline de ingesta: rate-limit por dispositivo, circuit-breaker, detección de gaps
- **server_ingest.rs:** Servidor MLLP TCP para recepción de monitores de cama
- **metrics.rs:** ~27 métricas Prometheus: HTTP, auth, clínicas, ML, HL7, ingest
- **observability.rs:** Health checks (/health, /live, /ready), graceful shutdown, reconnect DB
- **audit.rs:** Auditoría inmutable de acceso PHI con retención de 6 años
- **realtime.rs:** Hub SSE con broadcast, conexiones contadas, publicación de eventos
- **cache.rs:** Wrapper Valkey/Redis con inicialización global y fallback
- **middleware/:** Middleware de autenticación JWT (header o query param)

- **migrations.rs:** 3 migraciones SurrealQL: baseline, MFA/users, refresh tokens

4.3 Modulos del Frontend (33 archivos fuente)

- **app.rs:** Enrutador Leptos: 9 rutas protegidas, gate de autenticacion por signal
- **api.rs:** Cliente HTTP tipado: 40+ funciones, auth automatica via Bearer token
- **pages/login.rs:** Formulario con glassmorphism, error inline, estado de carga
- **pages/dashboard.rs:** Dashboard ejecutivo: 6 KPIs, graficos de dona, tarjetas de pacientes
- **pages/patients.rs:** Lista virtualizada con busqueda debounce 300ms, scrolling eficiente
- **pages/register.rs:** Formulario completo de ingreso: validacion en vivo con regex, picking de piel/ubicacion
- **pages/patient_detail.rs:** Expediente: scores, evolucion temporal, exportacion PDF/CSV
- **pages/measurement.rs:** Formulario de 5 escalas con computo en vivo en el navegador
- **pages/admin.rs:** 4 paneles: camas, equipos, staff, institucion
- **components/scales/:** 5 componentes de escala: Apache, GCS, NEWS2, SOFA, SAPS3
- **components/chart.rs:** Grafico de evolucion SVG: polilinea de APACHE con umbrales
- **components/radar_chart.rs:** Grafico radar SVG para visualizacion multi-escala
- **stores/realtime.rs:** SSE client con toast de alertas criticas automaticas
- **stores/theme.rs:** Persistencia dark/light mode en LocalStorage

4.4 Flujo de Datos Clinicos

El flujo completo de una medicion clinica es: (1) El medico/enfermero ingresa datos fisiologicos en el formulario de escala; (2) El frontend calcula el score en tiempo real usando la misma libreria Rust que el backend (compiled to WASM); (3) Al guardar, los datos viajan al backend via HTTP POST; (4) El backend re-calcula el score, valida rangos clinicos, persiste la medicion, actualiza los scores del paciente, y publica un evento SSE; (5) Todos los clientes conectados reciben la actualizacion. Alternativamente, cuando un monitor de cama envia datos via HL7 MLLP, el parser extrae las vitales, las clasifica por LOINC/mnemonico, y ejecuta el mismo pipeline.

5. Estado Actual de Desarrollo

5.1 Fases Completadas

Fase	Nombre	Estado	Fecha
0	Limpieza y cimientos (Rust 1.98, CI)	COMPLETADA	Sept 2026
1	Seguridad critica (JWT, MFA, RBAC)	COMPLETADA	Sept 2026
2	Arquitectura y datos (migraciones, transaccion	COMPLETADA	Sept 2026
3	DevOps y observabilidad (Prometheus, Grafan	COMPLETADA	Sept 2026
4	Frontend y UX (PWA, dark mode, SSE)	COMPLETADA	Sept 2026
5	Clinico y QA (FHIR, HL7, ML, E2E, proptests)	COMPLETADA	Sept 2026
6	Hardening y CI/CD	EN PROGRESO	Sept 2026
7	ML y Analytics v2	PLANIFICADA	Pendiente
8	Deployment y Ops (k8s, HIPAA)	PLANIFICADA	Pendiente

5.2 Especificaciones (SDD)

De 31 especificaciones formales (SPEC-001 a SPEC-031), 10 estan completadas, 2 estan listas para ejecucion, y 19 estan pendientes o congeladas. Las completadas incluyen: WASM build, persistencia ML, tests HL7, auth hardening, Prometheus/Grafana, Docker multi-stage, CI pipeline, alertas operativas, backup automatico, y staging.

5.3 Desglose por Area Funcional

Area	Estado	Detalle
Pacientes	100%	CRUD completo, busqueda, ingreso/egreso, expediente
Escalas clini.	95%	5 escalas calculadas, validacion, breakdown; pendiente validacion con vectores Knaus
Monitores HL7	92%	Parser ORU, MLLP TCP, ingest, rate-limit, circuit breaker
FHIR R4	90%	Patient, Condition, Observation, DiagnosticReport + QR
ML Prediccion	85%	Decision Tree entrenado, persistido; pendiente ensemble y explainability
Dashboard	95%	6 KPIs, graficos, tarjetas de pacientes, alertas en tiempo real
Admin UCI	90%	Camas, equipos, staff, institucion; pendiente multi-tenant
Seguridad	95%	JWT+MFA+RBAC+audit+rate-limit; pendiente 0 advisories en CI
Infra Docker	85%	Multi-stage, healthcheck, staging; pendiente k8s, cluster DB
Testing	80%	136+ tests, fuzzing, proptests, k6; pendiente coverage gate CI

6. Modelos de Datos y Dominio Clínico

6.1 Entidad Principal: Paciente

La entidad Patient es el núcleo del dominio, con más de 50 campos organizados en categorías: identidad (nombre, cédula, historia clínica), procedencia (nacionalidad, estado, ciudad), ingreso hospitalario (fechas, tipo de admisión, diagnóstico), estado clínico (ventilación mecánica, procesos invasivos), scores (APACHE, GCS, SOFA, SAPS3, NEWS2, riesgo de mortalidad), y metadatos (timestamps, cama asignada).

6.2 Entidad Medicion (Measurement)

Cada Measurement contiene un registro clínico diario con: datos crudos (ApacheIIData con 40+ campos fisiológicos, GcsData con 3 componentes), scores calculados (Apache II total + breakdown, GCS, SOFA, SAPS3, NEWS2), nivel de severidad, riesgo de mortalidad, y notas del clínico. Al crearse, calcula automáticamente TODOS los scores usando algoritmos validados.

6.3 Entidades de Recursos UCI

- **Cama:** Número, tipo (General/Aislamiento/Pediatría/Coronaria/Quemados/Otro), estado (Libre/Ocupada/Mantenimiento/Limpieza), paciente asignado.
- **Equipo:** Nombre, tipo (Ventilador/Monitor/Computador/Bomba Infusión), marca, modelo, serial, estado (Activo/Mantenimiento/Inactivo/Reparación), cama asignada.
- **Usuario:** Username, hash Argon2id del password, rol (Admin/Médico/Enfermero/Viewer), nombre, estado activo/inactivo.
- **Diagnóstico:** Código CIE-10, descripción, categoría. Catálogo de 38 diagnósticos comunes en UCI pre-cargados.

6.4 Base de Datos: SurrealDB

SurrealDB opera en modo embebido (SurrealKV) sin necesidad de servidor externo. El esquema se gestiona con 3 migraciones versionadas en SurrealQL que crean 11 tablas y 17 índices optimizados. Las operaciones críticas (ingreso de paciente con asignación de cama y equipos) usan transacciones atómicas con rollback automático. Las agregaciones GROUP BY se usan para estadísticas sin cargar filas completas a memoria.

7. Sistemas de Puntuacion Clinica

dMart implementa 5 sistemas de puntuacion clinica validados internacionalmente, todos con validacion de rangos fisiologicos, desglose por componente, y estimacion de mortalidad. Los algoritmos siguen estrictamente las publicaciones de referencia (Knaus 1985, Vincent 1996, Royal College of Physicians 2017, Moreno 2005).

Escala	Rango	Complejidad	Referencia	Rendimiento
APACHE II	0-71	12 vars + edad + cror	Knaus et al. 1985	~4 ns por calculo
GCS	3-15	3 componentes (ocular/verb	Teasdale & Jennett 1974	~1 ns
NEWS2	0-20+	7 signos vitales	Royal College Physicians 2017	~6 ns
SOFA	0-24	6 organos	Vincent et al. 1996	~6 ns
SAPS III	0-100	10+1/2/3 (edad, comorb.,	Moreno et al. 2005	~32 ns

7.1 Detalle: APACHE II (Acute Physiology and Chronic Health Evaluation)

APACHE II es la escala de severidad mas ampliamente usada en UCI a nivel mundial. Calcula un score de 0 a 71 sumando tres componentes: (1) APS (Acute Physiology Score): 12 variables fisiologicas (temperatura, presion arterial media, frecuencia cardiaca, frecuencia respiratoria, FiO2, PaO2, pH, sodio, potasio, creatinina, hematocrito, leucocitos) con puntos de 0 a 4 segun la distancia a valores normales; (2) Puntos por edad (0 a 6 puntos); (3) Puntos por enfermedad cronica (0 a 5 puntos). El score se clasifica en 4 niveles: Bajo (0-9, mortalidad <10%), Moderado (10-29, mortalidad 10-25%), Severo (20-29, mortalidad 25-50%), Critico (>=30, mortalidad >50%).

dMart calcula APACHE II con precision de nanosegundos (~4ns) tanto en el backend como en el frontend (via WASM), permitiendo retroalimentacion instantanea al clinico mientras ajusta los valores. Incluye desglose por componente (breakdown) y estimacion de mortalidad hospitalaria usando la tabla de referencia de Knaus 1985.

7.2 Validacion Clinica Pendiente (SPEC-028)

IMPORTANTE: Aunque las escalas siguen fielmente los algoritmos de referencia, aun no se han validado contra vectores de prueba oficiales (test vectors de Knaus/GCS/NEWS2). Esta validacion (SPEC-028) es un requisito obligatorio antes de declarar cualquier escala como apta para uso clinico real. Es una de las prioridades inmediatas del sprint.

8. Motor de Prediccion (Machine Learning)

8.1 Modelo Actual: Decision Tree

dMart incluye un modelo piloto de prediccion de mortalidad basado en un Arbol de Decision (Decision Tree) implementado con la libreria linfa (el equivalente Rust de scikit-learn). El modelo toma 14 features clinicos extraidos de los datos APACHE II y predice la probabilidad de mortalidad hospitalaria.

8.2 Features de Entrada (14 variables)

- Score APACHE II total
- GCS total (3-15)
- Edad del paciente
- Temperatura corporal
- Presion arterial media
- Frecuencia cardiaca
- Frecuencia respiratoria
- FiO2 (Fraccion inspirada de oxigeno)
- SpO2 (Saturacion de oxigeno)
- pH arterial
- Sodio serico
- Creatinina
- Leucocitos
- Potasio serico

8.3 Entrenamiento y Rendimiento

El modelo se entrena con 1,000 registros sinteticos generados usando reglas clinicas basadas en las tablas de referencia de APACHE II (distribuciones de mortalidad por rangos de score). Se evalua en un holdout de 200 registros con precision estimada de 85-90%. El modelo se persiste en disco usando bincode (formato binario eficiente) para evitar re-entrenamiento en cada reinicio del servidor.

8.4 Roadmap ML (Fase 7)

Las mejoras planificadas para el motor ML incluyen:

- **SPEC-014 — Feature Store:** Almacenamiento versionado de features clinicas en SurrealDB para reproducibilidad.
- **SPEC-015 — Ensemble:** Combinacion de Decision Tree + Logistic Regression + XGBoost para precision >92%.
- **SPEC-016 — A/B Testing:** Framework para comparar modelos en produccion con split de trafico.
- **SPEC-017 — SHAP Explainability:** Explicacion por-feature de cada prediccion para los

clínicos.

- **SPEC-018 — Retraining Pipeline:** Re-entrenamiento automático con detección de drift semanal.

9. Interoperabilidad (HL7 / FHIR)

9.1 HL7 v2 — Recepcion de Monitores

El sistema implementa un parser completo para mensajes HL7 v2 (formato ORU^R01), el estandar universal para la transmision de resultados de laboratorio y monitores clinicos. El parser soporta:

- Deteccion automatica del fabricante: Mindray (BeneVision/Benefit) y Philips (IntelliVue/MP5)
- Clasificacion de vitales via codigos LOINC y mnemonicos del fabricante
- Resolucion de pacientes por UUID (PID-18) o numero de expediente (PID-3)
- Transporte MLLP sobre TCP (puerto 2575 por defecto)
- Validacion de integridad del frame (START_BLOCK 0x0B, END_BLOCK 0x1C)

9.2 Pipeline de Ingesta (SPEC-031)

La ingesta de datos desde monitores incluye un pipeline de 11 pasos con proteccion contra fallos:

- Rate-limiting por dispositivo (token bucket configurable)
- Circuit breaker (abre despues de tasa de error alta, cierra automaticamente)
- Deteccion de gaps en secuencia (salto de numeracion = datos perdidos)
- Validacion de calidad de datos (rangos fisiologicos, campos obligatorios)
- Persistencia de estado en Valkey para sobrevivir reinicios

9.3 FHIR R4 — Interoperabilidad con Sistemas Externos

FHIR (Fast Healthcare Interoperability Resources) es el estandar moderno para el intercambio de datos clinicos digitales. dMart expone los siguientes recursos FHIR R4:

- **Patient:** Busqueda y lectura de pacientes en formato FHIR con codificacion CIE-10.
- **Condition:** Diagnosticos del paciente con codificacion CIE-10 en coding standard.
- **Observation:** Scores clinicos (GCS, APACHE II, SOFA) como observaciones FHIR.
- **DiagnosticReport:** Reporte clinico completo con representacion textual y codigo QR.
- **QR Code:** Generacion de QR PNG y SVG con datos del reporte clinico.

10. Seguridad y Cumplimiento Normativo

10.1 Autenticacion y Autorizacion

La capa de seguridad de dMart esta disenada para cumplir con estandares de software clinico (HIPAA Security Rule) y sigue principios de defensa en profundidad:

- **Hash de passwords: Argon2id** Algoritmo ganador del Password Hashing Competition. Configuracion por defecto: 19 MiB de memoria, dificultad ajustable via variables de entorno. Cada hash es unico y resistente a ataques de fuerza bruta y GPU.
- **Sesiones: JWT + Refresh Tokens rotativos** Access tokens de 15 minutos, refresh tokens de 7 dias en cookie HttpOnly + SameSite=Strict. Cada refresh rota el token y revoca el anterior. Reuso de un token comprometido revoca toda la familia de sesiones del usuario.
- **MFA: TOTP (Time-based One-Time Password)** Compatibel con Google Authenticator, Authy, etc. Incluye 10 codigos de respaldo alfanumericos de 8 caracteres. El MFA es opcional pero recomendado para cuentas con rol Medico o Admin.
- **RBAC: Control de acceso por rol** 4 roles con permisos granulares por ruta: Admin (acceso total), Medico (pacientes + mediciones), Enfermero (solo mediciones), Viewer (solo lectura). 100% de las rutas protegidas tienen un permiso mapeado en la tabla central de autorizacion.

10.2 Proteccion de Datos y Auditoria

- **Cifrado en reposo: ChaCha20-Poly1305** Cifrado autenticado (AEAD) con derivacion de clave via Argon2id KDF desde una clave maestra obligatoria (DMART_MASTER_KEY). El servidor se niega a arrancar si la clave es la por defecto.
- **Auditoria de acceso PHI:** Cada operacion sobre datos de pacientes genera un log inmutable con: usuario, accion, recurso, IP, user-agent, timestamp RFC 3339, exito/fallo. Los logs se retienen 6 anos (requisito HIPAA). Consultas de auditoria con filtros por usuario, accion, fecha y recurso.
- **Rate limiting:** Token bucket por IP real (no spoofeable via X-Forwarded-For). Throttle de login: bloqueo tras 5 intentos fallidos por IP. Headers de seguridad: HSTS, CSP, X-Frame-Options DENY, X-Content-Type-Options.
- **Zeroizacion de secretos:** Las passwords y tokens se borran de memoria despues de usarlas. Los hashes nunca se exponen en respuestas API (protegido en el modelo UserInfo).

10.3 Matriz de Permisos

La siguiente matriz resume los permisos por rol en el sistema:

Permiso	Admin	Medico	Enfermero	Viewer
Gestionar pacientes	SI	SI	NO	NO
Crear mediciones/escalas	SI	SI	SI	NO
Ver pacientes	SI	SI	SI	SI
Admin camas/equipos	SI	NO	NO	NO
Gestionar staff	SI	NO	NO	NO
Ver auditoria	SI	NO	NO	NO
Exportar datos	SI	SI	NO	NO

11. Infraestructura y Despliegue

11.1 Docker Multi-Stage

La imagen Docker de dMart usa un build multi-stage de 3 etapas para minimizar el tamaño final:

- Etapa 1 (builder-server): Compila el backend Rust con todas las dependencias
- Etapa 2 (builder-wasm): Compila el frontend a WebAssembly via cargo + wasm-bindgen
- Etapa 3 (runtime): Debian Bookworm Slim con solo los binarios necesarios (~48MB total)

11.2 Entornos de Despliegue

Entorno	Compose	Puerto	Estado
Desarrollo	docker-compose.yml	3000	Server + Valkey
Staging	docker-compose.staging.yml	3000	Validado end-to-end (R1/R4)
Produccion	docker-compose.prod.yml	3000	Caddy + server + Valkey

11.3 Produccion (docker-compose.prod.yml)

El entorno de produccion incluye: (1) Caddy como reverse proxy con TLS automatico (si se configura SITE_ADDRESS); (2) dmart-server como servicio principal con read_only del filesystem, drop de todas las capacidades Linux, limite de 1GB de memoria; (3) Valkey 8 como cache con LRU y 256MB de memoria; (4) 7 volúmenes named para persistencia de datos, config, y backups.

11.4 Monitoreo y Observabilidad

El sistema expone ~27 metricas Prometheus en /obs/metrics, incluyendo: latencia HTTP (buckets de 0.1ms a 10s), tasa de errores, metricas de autenticacion, metricas clinicas (patients, measurements, scales), metricas ML, metricas HL7, y metricas de ingesta. Estas alimentan 6 dashboards Grafana pre-configurados y 18 reglas de alerta Prometheus (validadas con promtool).

12. Calidad de Software y Testing

12.1 Estrategia de Testing (4 niveles)

dMart implementa una estrategia de testing en profundidad con 4 niveles complementarios:

- **Nivel 1 — Tests unitarios (136+ tests Rust):** Tests de libreria para escalas clinicas, validacion, HL7 parser, auth, RBAC, rate-limiting, y mas. Cada modulo critico tiene cobertura de tests.
- **Nivel 2 — Property-based testing (66 proptests):** Propiedades matematicas verificadas con datos aleatorios: monotonicidad de escalas (mayor fisiologia -> mayor score), limites (APACHE $\leq$ 71, SOFA $\leq$ 24), consistencia de breakdowns, robustez del parser HL7 ante entrada malformada.
- **Nivel 3 — Fuzzing (3 targets de cargo-fuzz):** Ingestion masiva de bytes aleatorios para encontrar crashes y panics: fuzz_api_json (endpoints JSON), fuzz_hl7_parser (parser HL7), fuzz_scales (calculadoras clinicas).
- **Nivel 4 — E2E y Load testing:** 15 tests Playwright (login -> pacientes -> mediciones -> admin) y 5 escenarios k6 con hasta 1000 requests/segundo para validar rendimiento bajo carga.

12.2 Metricas de Calidad

Metrica	Target	Actual	Estado
Tests Rust	>100	136+	CUMPLIDO
Prop-tests	>50	66	CUMPLIDO
Fuzz targets	>2	3	CUMPLIDO
E2E tests	>10	15	CUMPLIDO
Cobertura HL7	>90%	96.5%/94%/92%	CUMPLIDO
Clippy warnings	0	0	CUMPLIDO
cargo audit	0 criticos	0	CUMPLIDO
Cobertura total CI	>80%	Pendiente gate	PENDIENTE
Vectores clinicos	Validados	No validados	PENDIENTE

13. Que Falta para Produccion Hospitalaria

El sistema cuenta con un nucleo funcional solido, pero existen requisitos pendientes antes de su despliegue en una UCI real. Estos se organizan por prioridad:

13.1 Prioridad Alta (Requisitos de Piloto)

- **SPEC-028 — Validacion clinica con vectores de referencia:** Las escalas APACHE II, GCS, NEWS2, SOFA y SAPS III deben ser validadas contra los vectores de prueba oficiales de las publicaciones de referencia. Sin esto, ninguna escala se declara apta para uso clinico. ESTIMACION: 2 dias.
- **SPEC-027 — Coverage gate en CI:** Configurar cargo-llvm-cov como gate obligatorio en el pipeline CI para mantener cobertura de HL7 y escalas clinicas >90%. ESTIMACION: 0.5 dias.
- **SPEC-029 — Fingerprint y versionado de scores:** Cada calculo de score debe generar un hash reproducible con version del algoritmo para auditabilidad medico-legal. ESTIMACION: 2 dias.
- **SPEC-030 — Retencion de datos clinicos:** Implementar downsampling automatico de mediciones (raw -> hourly -> daily) para controlar el crecimiento de la base de datos. ESTIMACION: 2 dias.

13.2 Prioridad Media (Deuda Tecnica)

- **SPEC-010 — Semantic Versioning:** Configurar git-cliff para changelog automatico y tags semver. ESTIMACION: 0.5 dias.
- **SPEC-011 — WASM Optimizacion:** Aplicar wasm-opt para reducir el bundle de 2.2MB a ~600KB. ESTIMACION: 1 dia.
- **SPEC-013 — Web Push Notifications:** Notificaciones push VAPID para alertas de deterioro clinico. ESTIMACION: 2 dias.
- **Deuda tecnica Fase 3:** 6 items pendientes: alertas webhook/email, backup restore test, OpenTelemetry tracing, metricas de proceso. ESTIMACION: 3-5 dias.

13.3 Prioridad Baja (Pre-Produccion Hospitalaria)

- **SPEC-019 a SPEC-026 (congelados):** Kubernetes, GitOps, SurrealDB cluster, disaster recovery, HIPAA evidence pack, multi-tenancy, blue/green deploy. Estos quedan CONGELADOS hasta que exista un contrato o piloto hospitalario real, ya que no aportan valor a un piloto de 5 camas y suman

riesgo operativo.

- **ML Avanzado (SPEC-014 a SPEC-018):** Feature store, ensemble, A/B testing, SHAP explainability, retraining pipeline. Planificados para la Fase 7.

14. Analisis de Riesgos

14.1 Matriz de Riesgos

Riesgo	Prob.	Impacto	Mitigacion
Escalas con errores clinicos	Baja	Critico	SPEC-028: validacion con vectores de Knaus
Fallo de monitores HL7	Media	Alto	Circuit breaker + gap detection (SPEC-031)
Perdida de datos clinicos	Baja	Critico	Backup diario + retencion 30d (SPEC-009)
Ataque de seguridad	Baja	Critico	JWT+MFA+RBAC+audit+rate-limit+HSTS
Degradacion del rendimiento	Media	Medio	Rate-limit + cache + buckets finos
Drift del modelo ML	Media	Medio	SPEC-018: retraining pipeline con drift detection
Crecimiento de DB sin limites	Alta	Medio	SPEC-030: downsampling raw->hourly->daily
Fallo humano en configuracion	Media	Alto	Defaults seguros, validacion al arranque

14.2 Criterios Go/No-Go del Piloto

El acuerdo de arquitectura del 13 de septiembre de 2026 define que el piloto es 'robusto' cuando el sistema sobrevive 30 días en una UCI de 5 camas sin intervencion directa del desarrollador. Los 8 criterios medibles son:

- **R1:** 30 días de uptime sin intervencion manual
- **R2:** Detecta sus propios fallos (6 dashboards, 18 alertas activas, <1 falsa alarma/semana)
- **R3:** No pierde datos clinicos (backup diario + restauracion probada, RPO<=24h)
- **R4:** Despliegue y rollback sin drama (<15 min, imagen ~50MB)
- **R5:** Escalas clinicamente correctas (validadas contra vectores de referencia)
- **R6:** Un monitor roto no tumba el resto (circuit breaker, gap/fault detectados)
- **R7:** PHI y seguridad sin sorpresas (0 advisories, sin PHI en logs/metricas)
- **R8:** Cambios no rompen lo existente (CI gate: tests + clippy + promtool verdes)

15. Cronograma y Proyeccion

15.1 Sprint Actual (Proximas 2 semanas)

El sprint actual se enfoca en cerrar los requisitos criticos para el piloto (R1-R8). El orden de ejecucion recomendado es:

- **Semana 1:** SPEC-028 (vectores clinicos) -> SPEC-027 (coverage gate) -> SPEC-010 (semver)
- **Semana 2:** SPEC-011 (wasm-opt) -> SPEC-029 (fingerprint scores) -> SPEC-030 (retencion)

15.2 Fase 6 Restante (Sprints 2-4)

Las remaining items de Fase 6 incluyen: Web Push Notifications (SPEC-013), deuda tecnica de Fase 3 (alertas webhook, OpenTelemetry, metricas de proceso), y refinamiento del pipeline CI.

15.3 Proyeccion a Largo Plazo

Fase	Contenido	Dependencia	Estimacion
6	Hardening + CI/CD completo	Ninguna (en curso)	2-3 semanas
7	ML avanzado (ensemble, SHAP)	SPEC-002 completado	4-6 semanas
8	k8s, GitOps, cluster, HIPAA	Contrato hospitalario	3-4 meses
Piloto	UCI real, 5 camas, 30 dias	Fase 6 + R5	Inmediato post-6

16. Conclusiones y Recomendaciones

16.1 Síntesis del Estado del Proyecto

dMart UCI ha alcanzado un nivel de madurez significativo con las primeras 6 fases completadas. El sistema cuenta con un núcleo funcional sólido que incluye:

- Cálculo clínico de 5 escalas internacionales en nanosegundos
- Predicción de mortalidad con Machine Learning
- Ingesta automática desde monitores vía HL7 v2
- Interoperabilidad FHIR R4 con generación de reportes y QR
- Seguridad de grado hospitalario (JWT + MFA + RBAC + audit)
- Frontend moderno compilado a WebAssembly con soporte offline
- Infraestructura Docker con monitoreo Prometheus + Grafana
- 136+ tests automatizados con estrategia de 4 niveles

16.2 Brechas Restantes

Las brechas principales para llegar a producción hospitalaria son: (1) Validación clínica formal de las escalas contra vectores de referencia; (2) Optimización del tamaño WASM para hardware clínico de gama baja; (3) Coverage gate en CI para mantener la calidad; (4) Auditabilidad médico-legal de los scores (fingerprint + versionado); (5) Estrategia de retención de datos a largo plazo.

16.3 Recomendaciones para los Asesores

- **Prioridad inmediata — Validación clínica:** Se recomienda encarecidamente completar la SPEC-028 (validación contra vectores de Knaus/GCS/NEWS2) ANTES de cualquier piloto clínico. Un score incorrecto tiene consecuencias directas sobre decisiones de tratamiento.
- **Estrategia de datos a largo plazo:** El sistema generará ~10,000 registros de medición por cama por año. Para una UCI de 20 camas, esto implica ~200,000 registros/año. La estrategia de downsampling (SPEC-030) es esencial para la sostenibilidad.
- **ML como herramienta de apoyo, no de decisión:** El modelo actual (85-90% precisión) es un indicador de apoyo. Las decisiones clínicas deben seguir siendo tomadas por el equipo médico. La Fase 7 (ensemble + SHAP) buscará precisión >92% y explicabilidad.
- **Protección de datos desde el día 1:** La auditoría PHI con retención de 6 años ya está implementada. Se recomienda activar las políticas de backup y revisar los logs de auditoría semanalmente durante el piloto.

16.4 Viabilidad Técnica

Desde la perspectiva técnica, el proyecto está en excelente estado. La selección de Rust como lenguaje garantiza seguridad de memoria y rendimiento nativo. La arquitectura modular permite evolucionar por componentes. La estrategia de testing en 4 niveles (unit, property, fuzz, E2E) ofrece una red de seguridad robusta. El uso de especificaciones formales (SDD) asegura trazabilidad entre requisitos y código. El sistema está listo para iniciar un piloto clínico controlado de 5 camas, una vez completados los ítems críticos restantes (SPEC-028, SPEC-029, SPEC-027).

Fin del Informe Tecnico

Generado el 14/09/2026 a las 10:52
dMart UCI v0.5.0 — Universidad Central de Venezuela
Documento confidencial para uso academico interno

Todos los metricas de rendimiento fueron medidos con cargo bench en hardware estandar.

Los algoritmos clinicos siguen las publicaciones de referencia citadas en el texto.

El codigo fuente esta disponible en: <https://github.com/roosvelt6/dmart>